Smart Parking System

Parking slot availability and entry control

Team 8

- JETHENDRA VARDHAN — 2620030600
- LAXMI PRAGYA — 2620030602
- VEDA CHANDRA — 2620030606
- CHANDRA PAUL — 2620030636
- SRIKRISHNA — 2620030675

Project Overview

- Uses sensor inputs to check slot availability
- Uses Boolean logic to control entry
- Shows allow or deny status instantly

DDCA PROJECT | DIGITAL DESIGN | BOOLEAN LOGIC

Made with GAMMA

The Problem & The Digital Solution

The Problem

- Drivers can’t see free slots before entering
- Cars still try to enter when full
- Manual checks are slow and error-prone
- No real-time slot or gate control

The Digital Solution

- Turn slot status into **binary signals**
- Use **logic gates** to read inputs in real time
- Detect availability automatically
- Allow entry only when **available AND requested**
- Show a live **PARKING_FULL** signal

Sensor Convention — Binary Input Definitions

Signal	Name	0	1
S1	Slot 1	Free	Occupied
S2	Slot 2	Free	Occupied
S3	Slot 3	Free	Occupied
S4	Slot 4	Free	Occupied
E	Entry Request	No	Yes

Key principle: 0 = Free, 1 = Occupied.

Made with GAMMA

Inputs, Binary States & Parking Conditions

Parking Slot Sensors

S1 — Slot 1

1 = Occupied | **0** = Free

S2 — Slot 2

1 = Occupied | **0** = Free

S3 — Slot 3

1 = Occupied | **0** = Free

S4 — Slot 4

1 = Occupied | **0** = Free

Entry Request

E — Entry Request

1 = Request | **0** = No request

NOT Gate Conversion

Invert each slot signal:

Signal	Meaning	Sn=0	Sn=1
¬S1	Slot 1 free	1	0
¬S2	Slot 2 free	1	0
¬S3	Slot 3 free	1	0
¬S4	Slot 4 free	1	0

1 means the slot is free. The OR gate checks overall availability.

Made with GAMMA

Logic Gates — Sensors to Entry

Stage 1 — NOT

- `S1 = 1` → `¬S1 = 0`
- `S1 = 0` → `¬S1 = 1`
- Same for S2, S3, and S4

Stage 2 — OR

Checks if **any** slot is free:

> **AVAILABLE = ¬S1 + ¬S2 + ¬S3 + ¬S4**

Stage 3 — AND

Entry needs **availability** and **request**:

> **ENTRY_ALLOWED = AVAILABLE · E**
> **= (¬S1 + ¬S2 + ¬S3 + ¬S4) · E**

Stage 4 — Full Parking

Parking is full when all slots are occupied:

> **PARKING_FULL = ¬AVAILABLE = S1 · S2 · S3 · S4**

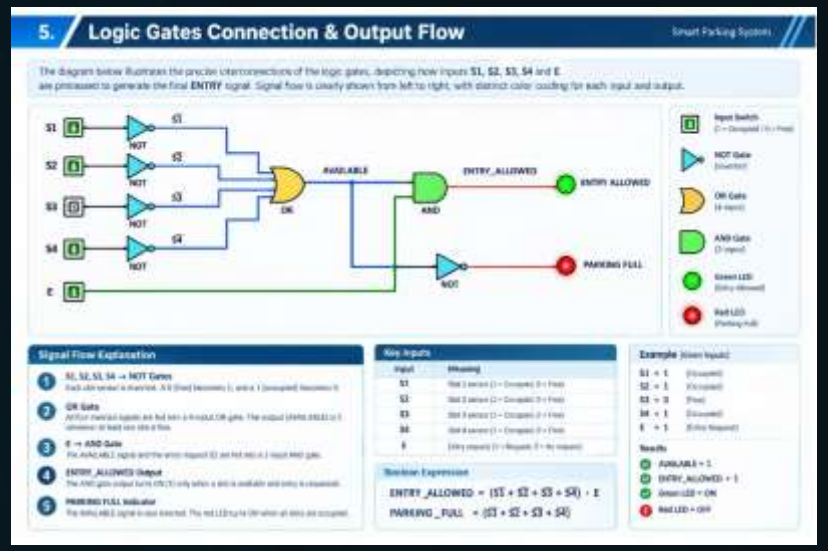
Gate Summary

Gate	Inputs	Output	Use
NOT ×4	S1–S4	¬S1–¬S4	Invert
OR (4-in)	¬S1–¬S4	AVAILABLE	Find free slot
AND (2-in)	AVAIL, E	ENTRY_ALLOWED	Allow entry
NOT (1-in)	AVAILABLE	PARKING_FULL	Full alert

No memory, no clock — just current inputs.

Made with GAMMA

Logic Gates Connection & Output Flow



Circuit Operation — Left to Right

1 Sensor Inputs

S1–S4 enter as **1 = Occupied, 0 = Free**

2 NOT Gates

Invert each sensor: `¬S1, ¬S2, ¬S3, ¬S4`

3 OR Gate

`AVAILABLE = ¬S1 + ¬S2 + ¬S3 + ¬S4`

4 AND Gate + Full Signal

```
ENTRY_ALLOWED = AVAILABLE · E
PARKING_FULL = ¬AVAILABLE
```

Worked Examples

Example 1: `S1=1, S2=1, S3=0, S4=1, E=1`

- AVAILABLE = 1
- ENTRY_ALLOWED = 1
- **ENTRY ALLOWED ✓**

Example 2: `S1=1, S2=1, S3=1, S4=1, E=1`

- AVAILABLE = 0
- PARKING_FULL = 1
- ENTRY_ALLOWED = 0
- **ENTRY DENIED ✗**

Made with GAMMA

Simulation & Decision Logic

Step-by-Step Process

01

Read Sensors

Get S1, S2, S3, S4

02

Invert Signals

Make ¬S1, ¬S2, ¬S3, ¬S4

03

Find Availability

`AVAILABLE = ¬S1 + ¬S2 + ¬S3 + ¬S4`

04

Check Request

Read input `E`

05

Decide Entry

`ENTRY_ALLOWED = AVAILABLE · E`

Simulation Cases

Case A — Spot Free, Entry On

`S1=1, S2=0, S3=1, S4=1, E=1`

- AVAILABLE = 1
- ENTRY_ALLOWED = **1**

ENTRY PERMITTED ✓

Case B — Parking Full

`S1=1, S2=1, S3=1, S4=1, E=1`

- AVAILABLE = **0**
- ENTRY_ALLOWED = **0**
- PARKING_FULL = **1**

ENTRY DENIED ✗

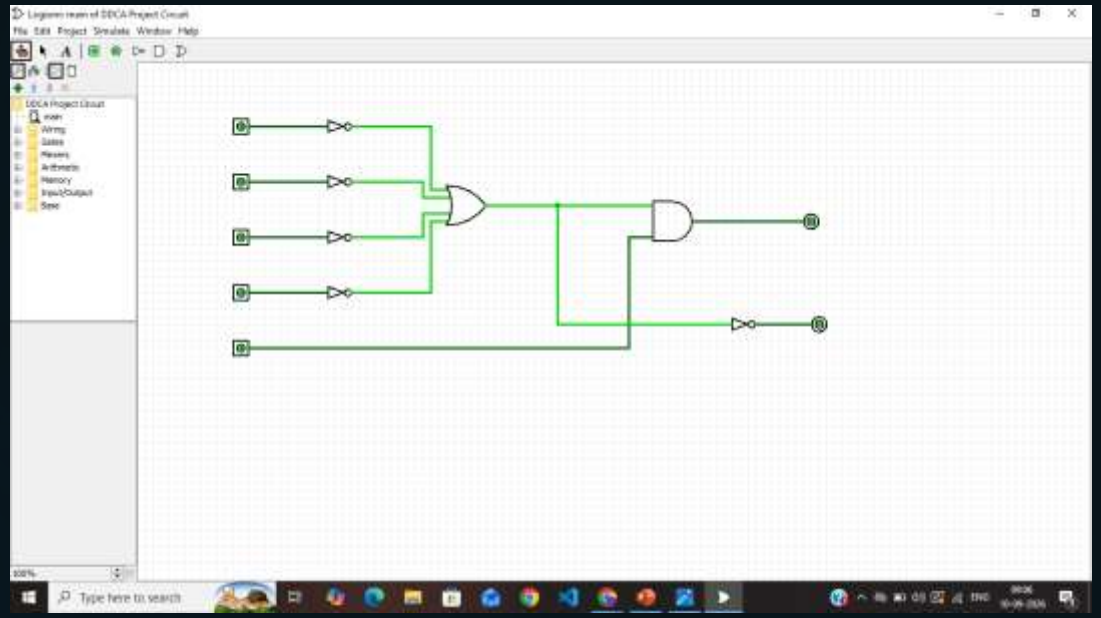
Case C — Spot Free, No Entry

`S1=1, S2=0, S3=1, S4=1, E=0`

- AVAILABLE = 1
- ENTRY_ALLOWED = **0**

NO ENTRY

E must be 1 to allow entry.



Made with GAMMA

Truth Table & Logic Verification

Simplified Truth Table — Parking Logic

Parking Condition	AVAILABLE	E	ENTRY_ALLOWED	PARKING_FULL
One slot free	1	0	0	0
One slot free	1	1	1	0
All slots full	0	0	0	1
All slots full	0	1	0	1

ENTRY_ALLOWED = 1 only when AVAILABLE=1 and E=1.

Logic Checks

Free + Request

AVAILABLE=1 and E=1 → ENTRY_ALLOWED=1.

Full Parking

AVAILABLE=0 → ENTRY_ALLOWED=0.

No Request

E=0 → no entry.

No Memory

Output depends only on current inputs.

Made with GAMMA

Conclusion

Project Summary

4 Binary Sensors

S1–S4 show whether each parking slot is occupied or free.

NOT Gates

Convert occupied slots into free-slot signals: ¬S1, ¬S2, ¬S3, ¬S4.

OR Gate

```
AVAILABLE = ¬S1 + ¬S2 + ¬S3 + ¬S4
```

AND Gate

```
ENTRY_ALLOWED = AVAILABLE · E
```

Full Parking

```
PARKING_FULL = S1·S2·S3·S4
```

Key Design Features

Verified

Matches the truth table.

Scalable

Add more slots by expanding the logic.

Combinational

No clock or memory.

Design Flow

1. BITS
2. LOGIC
3. CIRCUIT
4. DECISION

Made with GAMMA